



Facultad de Ingeniería

Maestría en Inteligencia Artificial (MAIA)

Aprendizaje por Refuerzo

Entrega parcial

Ambiente, estados y acciones

Juan David Lara Camacho

Agustín Serrano

Universidad de los Andes

6 de mayo de 2026

Resumen

Modelamos el ambiente `MiniGrid-BlockedUnlockPickup-v0` como un proceso de decisión de Markov determinista para entrenar un agente Q-learning tabular. La tarea exige una secuencia obligatoria de subtareas: mover una bola que bloquea la puerta, recoger la llave, abrir la puerta, soltar la llave y, por último, recoger la caja objetivo. La representación del estado tiene que capturar tanto la posición y orientación del agente como el progreso ya alcanzado en el episodio. Definimos el estado con ocho componentes, las seis acciones discretas que ofrece MiniGrid y una función de recompensa con *shaping* por subtareas que suma a lo más +2.80 por episodio. Con esta formulación el agente alcanza 100/100 de éxito en evaluación greedy en 24 pasos.

1. Introducción

El proyecto del curso pide modelar y resolver un problema con aprendizaje por refuerzo siguiendo el marco estándar de los procesos de decisión de Markov [1]. Esta entrega cubre la primera parte: la definición formal del ambiente. La rúbrica pide tres cosas:

1. el conjunto de estados $\mathcal{S}$ que captura toda la información relevante para decidir,
2. el conjunto de acciones $\mathcal{A}$ y el comportamiento esperado del agente,
3. el modelo de recompensa expresado como tripletas $(s, a, s') \mapsto r$.

Para esta entrega elegimos modelar el ambiente `MiniGrid-BlockedUnlockPickup-v0` de la librería MiniGrid [3]. Su estructura de subtareas (mover una bola que bloquea una puerta, recoger una llave, abrir la puerta, soltar la llave y recoger una caja objetivo) obliga a un diseño de estado y de recompensa con varias variables auxiliares, dependencias de orden entre subtareas y *shaping* de la señal de aprendizaje. El resto del documento desarrolla formalmente la definición del ambiente sobre el que entrenamos el agente.

2. Ambiente: MiniGrid-BlockedUnlockPickup-v0

El ambiente es una grilla rectangular con dos cuartos separados por un muro vertical y conectados por una sola casilla con puerta. El agente parte en el cuarto izquierdo y debe llegar a una casilla objetivo en el cuarto derecho. La puerta está cerrada y necesita una llave del mismo color. Además, hay una bola del mismo color que la puerta justo enfrente, bloqueando el acceso. La Figura 1 muestra el esquema general del ambiente y la Figura 2 cinco fotogramas de un episodio greedy ya resuelto.

Dos detalles del ambiente subyacente (la implementación `MiniGrid-BlockedUnlockPickup-v0`) conviene fijarlos desde ahora, porque condicionan el conjunto de acciones y la condición de terminación que aparecen más adelante. Primero, retirar la bola que bloquea la puerta se hace recogiénola con `pickup` y soltándola en otra celda con `drop`: en MiniGrid no existe una acción de empujar, así que la única forma de despejar la bola es usar el inventario (que admite un solo objeto a la vez). Segundo, la casilla objetivo del cuarto derecho está materializada en el ambiente como la celda donde se encuentra la caja (`box`). El ambiente dispara `terminated=True` cuando el agente la recoge. Por eso nuestro modelo incluye `drop` entre las acciones y usa `box_picked` como condición terminal.

La tarea no se puede resolver sin completar las subtareas en un orden fijo: mover la bola, recoger la llave, abrir la puerta, recoger la caja. El ambiente no admite atajos. La Figura 3 formaliza esa secuencia.

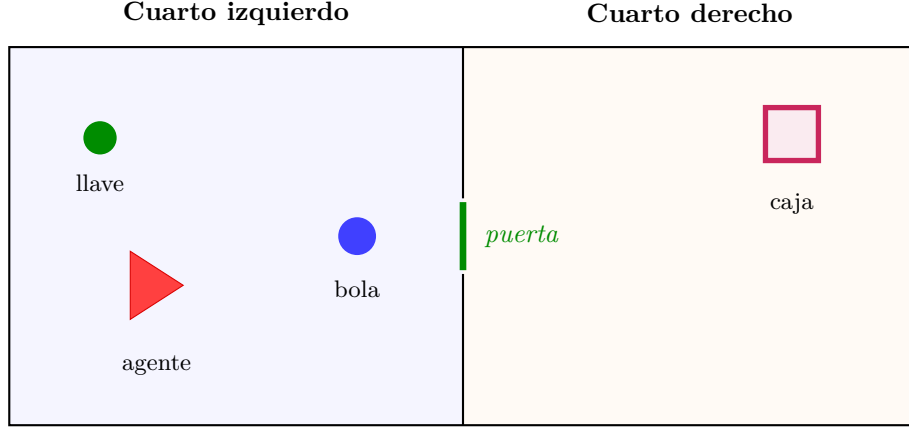


Figura 1: Esquema del ambiente. Dos cuartos separados por un muro vertical con una sola casilla de paso (la puerta). Las dimensiones exactas, los anchos de cuarto y las posiciones iniciales de los objetos se aleatorizan al reiniciar el ambiente; fijamos la semilla del generador para que cada ejecución arranque desde la misma configuración. El color de la puerta y de la llave también se aleatoriza por episodio: en nuestra semilla salieron verdes (los frames de la Figura 2 muestran esa misma ejecución).

Algunos parámetros del ambiente que conviene fijar de entrada. El horizonte máximo de un episodio es `max_steps= 576`, valor por defecto del ambiente. Las orientaciones se codifican como enteros 0, 1, 2, 3 para Este, Sur, Oeste y Norte. El ambiente es completamente determinista una vez fijada la semilla del generador.

3. Conjunto de estados

3.1. Definición formal

Cada estado se representa como una tupla de ocho componentes:

$$s = (\underbrace{(x, y)}_{\text{pos}}, \underbrace{d}_{\text{dir}}, \underbrace{c}_{\text{carrying}}, \underbrace{b}_{\text{ball_moved}}, \underbrace{k_p}_{\text{key_picked}}, \underbrace{o}_{\text{door_open}}, \underbrace{k_d}_{\text{key_dropped}}, \underbrace{x_p}_{\text{box_picked}}) \quad (1)$$

La Tabla 1 describe cada componente.

Componente	Tipo	Rango	Descripción
<code>pos</code>	$(\mathbb{Z}_{\geq 0})^2$	$(0..W-1, 0..H-1)$	posición del agente en la grilla
<code>dir</code>	$\mathbb{Z}$	$\{0, 1, 2, 3\}$	orientación: 0 E, 1 S, 2 W, 3 N
<code>carrying</code>	cadena	$\{\text{none}, \text{key}, \text{ball}, \text{box}\}$	objeto que el agente lleva en el momento actual
<code>ball_moved</code>	booleano	$\{0, 1\}$	la bola ya no está en su posición inicial
<code>key_picked</code>	booleano	$\{0, 1\}$	el agente recogió la llave en algún momento del episodio
<code>door_open</code>	booleano	$\{0, 1\}$	la puerta está abierta
<code>key_dropped</code>	booleano	$\{0, 1\}$	el agente soltó la llave después de abrir la puerta
<code>box_picked</code>	booleano	$\{0, 1\}$	el agente cargó la caja al menos una vez

Tabla 1: Componentes del estado. Las primeras cinco filas son lo que el agente puede observar directamente del ambiente. Las tres últimas son banderas de progreso que necesitamos para que la función de recompensa sea Markoviana sobre (s, a, s') ; ver Sección 6.

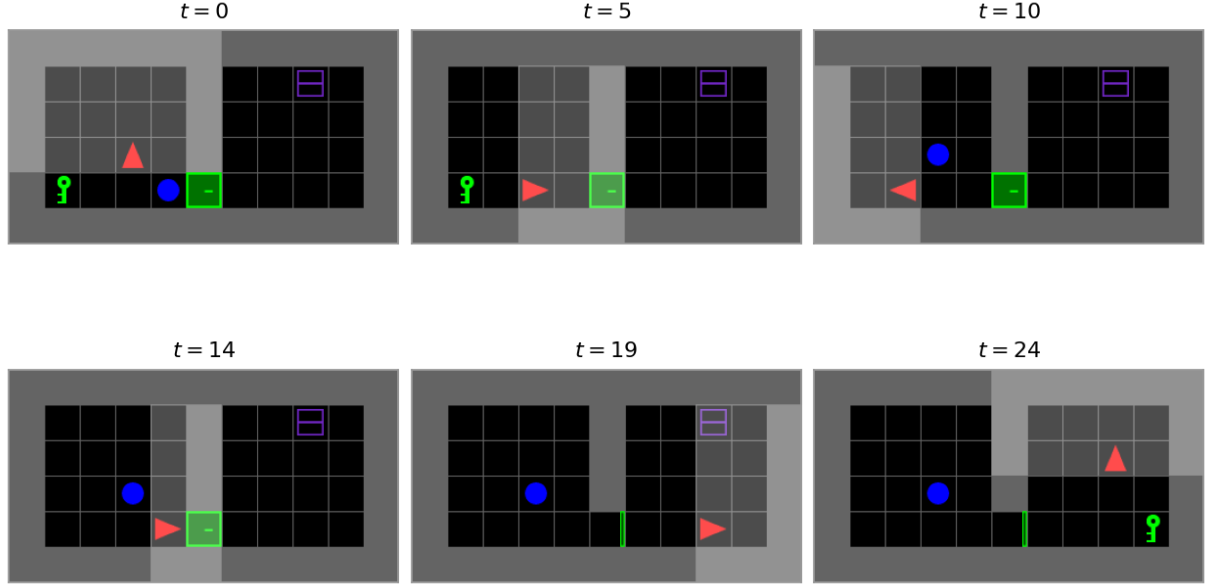


Figura 2: Seis fotografías equiespaciadas de un episodio greedy del agente entrenado, en orden de izquierda a derecha y de arriba a abajo. La fila superior cubre la fase en el cuarto izquierdo: en $t = 0$ el agente parte desde su posición inicial, en $t = 5$ la bola ya fue retirada de enfrente de la puerta y en $t = 10$ el agente carga la llave camino al umbral. La fila inferior cubre la fase en el cuarto derecho: en $t = 14$ la puerta está abierta y el agente la cruza, en $t = 19$ avanza hacia la caja y en $t = 24$ la recoge, lo que termina el episodio.

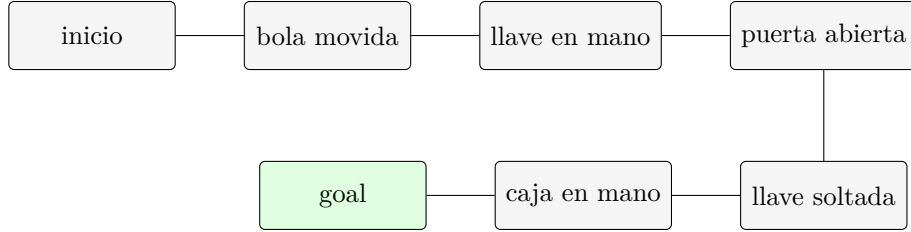


Figura 3: Cadena de subtareas que el agente debe completar. La fila superior se lee de izquierda a derecha y la fila inferior de derecha a izquierda, siguiendo el orden temporal de los eventos. Cada conexión representa el cambio que se produce la primera vez que una acción modifica ese aspecto del estado dentro del episodio.

3.2. Justificación

Las primeras cinco componentes son las obvias. La posición y la orientación fijan qué celda mira el agente. El componente **carrying** indica qué tiene en mano. Importa porque la acción **pickup** solo funciona con las manos vacías. También importa qué objeto se está cargando, porque eso restringe lo que se puede hacer a continuación. Las banderas **ball_moved** y **door_open** son condiciones del ambiente que el agente podría inducir mirando las celdas vecinas. Conviene exponerlas explícitamente para que la Q-tabla las indexe sin ambigüedad.

Las tres últimas (**key_picked**, **key_dropped** y **box_picked**) parecen redundantes a primera vista: **key_picked** se podría deducir de **carrying = key** en el momento del **pickup**, y **key_dropped** de la combinación de variables anteriores. Las incluimos por la *recompensa*, no por la representación del progreso físico. Nuestro *shaping* entrega cada bonificación una sola vez por episodio: la primera vez que se alcanza la condición. Sin estas banderas, la misma transición (s, a, s') podría producir recompensas distintas según la historia previa del episodio. La Sección 6 desarrolla este punto.

3.3. Tamaño del espacio

Una cota superior, tomando todos los productos cartesianos posibles, es $W \cdot H \cdot 4 \cdot 4 \cdot 2^5$. Para la configuración por defecto, con grilla aproximadamente 11×6 , eso da del orden de 10^4 estados. La cota es laxa. Las restricciones de consistencia eliminan la mayor parte de las combinaciones: `key_dropped = 1` implica `door_open = 1` y `key_picked = 1`, y `box_picked = 1` implica las tres banderas anteriores. En 10 000 episodios de entrenamiento el agente visita 635 estados distintos.

3.4. Estado inicial y estados terminales

El estado inicial s_0 tiene posición y orientación fijas (las del `reset()` con la semilla del experimento), `carrying = none` y todas las banderas en cero. Un estado se vuelve terminal cuando `box_picked = 1`: el ambiente devuelve `terminated=True` y el agente recibe la recompensa terminal. Si en cambio se alcanza `max_steps` sin éxito, el ambiente devuelve `truncated=True`; ese caso no se considera terminal en el sentido del MDP, simplemente cierra el episodio por límite de tiempo.

4. Conjunto de acciones

El conjunto de acciones es $\mathcal{A} = \{0, 1, 2, 3, 4, 5\}$ y corresponde a las seis acciones discretas que MiniGrid expone para este tipo de tareas [3]. Cada acción está definida en todo el espacio de estados, pero solo produce un cambio efectivo de estado cuando se cumplen ciertas condiciones locales. La Tabla 2 indica, para cada acción, los estados en los que es aplicable y el efecto que produce.

Idx	Nombre	Aplicable en	Efecto
0	<code>left</code>	todos los estados	$d \mapsto (d - 1) \bmod 4$
1	<code>right</code>	todos los estados	$d \mapsto (d + 1) \bmod 4$
2	<code>forward</code>	estados con celda al frente transitable (vacía o puerta abierta)	avanza una celda en dirección d
3	<code>pickup</code>	estados con <code>carrying = none</code> y un objeto recogible (llave, bola o caja) en la celda al frente	<code>carrying</code> toma el tipo del objeto, la celda queda vacía
4	<code>drop</code>	estados con <code>carrying $\neq$ none</code> y celda al frente vacía	deja el objeto en esa celda, <code>carrying</code> pasa a <code>none</code>
5	<code>toggle</code>	estados con la puerta al frente y <code>carrying = key</code>	abre la puerta: <code>door_open</code> pasa de 0 a 1

Tabla 2: Acciones disponibles, con los estados en los que cada una es aplicable y el efecto resultante. Si una acción se ejecuta en un estado donde no es aplicable, el ambiente la admite sin error pero deja $s' = s$; solo avanza el contador de pasos.

Mantenemos las seis acciones disponibles en todos los estados, en lugar de podar dinámicamente las que no son aplicables. El agente tiene que aprender por sí mismo cuándo cada acción tiene efecto y cuándo no. El costo por paso penaliza las acciones inútiles, así que esa información se filtra hacia la política sin que tengamos que codificarla a mano.

5. Modelo de transición

El ambiente es determinista una vez fijada la semilla: $P(s' \mid s, a) \in \{0, 1\}$. Existe una única función $T : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ tal que $s' = T(s, a)$. La Tabla 3 muestra una selección de transiciones representativas. Todas incluyen el costo de paso -0.001 en la columna r ; los demás componentes se desglosan en la Sección 6.

Estado s	Acción a	Estado s'	r
libre, casilla del frente vacía	forward	misma config., pos avanzada	-0.001
de frente a un muro	forward	idéntico a s (no-op)	-0.001
cualquier estado	left	idéntico salvo $d-1$ mód 4	-0.001
de frente a la bola, manos vacías, $b=0$	pickup	carrying=ball , $b=1$	$-0.001 + 0.20$
cargando bola, casilla del frente vacía	drop	carrying=none , bola en esa celda	-0.001
de frente a la llave, manos vacías, $k_p=0$	pickup	carrying=key , $k_p=1$	$-0.001 + 0.30$
de frente a la puerta, llave en mano, $o=0$	toggle	$o=1$	$-0.001 + 0.50$
de frente a la puerta, sin llave	toggle	idéntico a s (no-op)	-0.001
puerta abierta, llave en mano, casilla vacía, $k_d=0$	drop	carrying=none , $k_d=1$	$-0.001 + 0.30$
de frente a la caja, manos vacías, $x_p=0$	pickup	carrying=box , $x_p=1$, terminal	$-0.001 + 0.50 + 1.00$
de frente a la bola, ya $b=1$	pickup	se recoge la bola pero <i>no</i> repite el bono	-0.001
estado terminal alcanzado	cualquier	el episodio se ha cerrado	n/a

Tabla 3: Transiciones representativas. Cubren las clases relevantes: movimiento libre, choque, giro, las cinco subtareas en su primera ocurrencia, una repetición sin bono y un **toggle** sin llave.

6. Función de recompensa

La función $R(s, a, s')$ suma tres ingredientes: un costo de paso constante, las bonificaciones de *shaping* por subtareas y la recompensa terminal del ambiente subyacente:

$$R(s, a, s') = \text{step_cost} + \sum_{\ell \in \mathcal{L}} \beta_{\ell} \Delta_{\ell}(s, s') + r_{\text{goal}} \mathbb{I}[s' \text{ terminal}], \quad (2)$$

donde $\mathcal{L}$ es el conjunto de cinco subtareas y β_{ℓ} la bonificación asociada (Tabla 4). El indicador $\Delta_{\ell}(s, s')$ vale 1 cuando la bandera correspondiente pasa de 0 en s a 1 en s' , y 0 en cualquier otro caso. La construcción de Δ_{ℓ} usa las banderas **key_picked**, **key_dropped** y **box_picked** dentro de s y s' . Por eso esas banderas son parte del estado, como anticipamos en la Sección 3.

6.1. Por qué *shaping*

La recompensa nativa del ambiente vale ~ 1 cuando se entrega la caja y 0 en cualquier otro paso. Para Q-learning tabular [2] eso es problemático. Mientras el agente no logre por azar toda la cadena de subtareas en un mismo episodio, no recibe ninguna señal positiva, y los Q-valores no se diferencian entre trayectorias. En las primeras pruebas con la recompensa cruda el agente entrenó 10 000 episodios sin encontrar la caja una sola vez. Por eso introducimos las cinco bonificaciones

Componente	Valor	Condición
<code>step_cost</code>	-0.001	cada paso, sin excepción
β_{ball}	+0.20	$b : 0 \rightarrow 1$
$\beta_{\text{key_p}}$	+0.30	$k_p : 0 \rightarrow 1$
β_{door}	+0.50	$o : 0 \rightarrow 1$
$\beta_{\text{key_d}}$	+0.30	$k_d : 0 \rightarrow 1$ (requiere $o = 1$ y $k_p = 1$)
β_{box}	+0.50	$x_p : 0 \rightarrow 1$
r_{goal}	+1.00	s' terminal (caja entregada)

Tabla 4: Componentes de la recompensa. La suma máxima por episodio, alcanzada cuando el agente completa todas las subtareas exactamente una vez y resuelve el episodio en N pasos, vale $+0.20 + 0.30 + 0.50 + 0.30 + 0.50 + 1.00 + N \cdot (-0.001) = 2.80 - 0.001 N$.

intermedias. Cada una marca un hito intermedio del problema. Con esos hitos la Q-tabla recibe señal útil mucho antes de que el agente, por azar, complete una trayectoria exitosa entera.

6.2. Por qué los valores específicos

Buscamos que las bonificaciones intermedias no dominaran a la recompensa terminal y que el costo por paso no las anulara. Con `max_steps=576`, un costo de paso de -0.01 acumula -5.76 en un episodio largo, lo que borra la suma de los β_ℓ . Por eso bajamos el costo a -0.001 . Las bonificaciones se distribuyeron cualitativamente según la dificultad percibida de cada subtask (0.20 para mover la bola, 0.50 para abrir la puerta) y se ajustaron de forma empírica hasta lograr convergencia consistente.

6.3. Por qué solo la primera vez

Sin la restricción *first-time-only*, el agente podría inflar su retorno haciendo `pickup` y `drop` de la llave varias veces seguidas. Las banderas `key_picked`, `key_dropped` y `box_picked` dentro del estado evitan ese *bonus farming*: una vez que la bandera entra en 1, Δ_ℓ vale 0 por mucho que se repita la acción.

7. Consideraciones especiales

7.1. Determinismo y reproducibilidad

Tanto `env.reset(seed=0)` como `random.seed(42)` en el agente quedan fijos. Eso hace que el experimento sea repetible: dos corridas distintas producen exactamente la misma trayectoria y la misma Q-tabla. La configuración inicial del ambiente (posiciones de los objetos, color de la puerta) también queda determinada por la semilla.

7.2. Resultados resumidos

Aunque la rúbrica de esta entrega no lo pide, dejamos los números de la evaluación greedy del agente entrenado. Sirven para verificar que la formulación es implementable. Sobre 100 episodios de evaluación, el agente alcanza tasa de éxito 100/100, recompensa promedio $+2.776$ y 24 pasos en promedio. La curva de aprendizaje (Figura 4) muestra las tres fases típicas. La primera es una exploración prácticamente plana hasta unos 2 000 episodios. La segunda es de aprendizaje rápido,

entre 2000 y 4000 episodios. La tercera es de convergencia oscilante alrededor del máximo teórico +2.80 de allí en adelante.

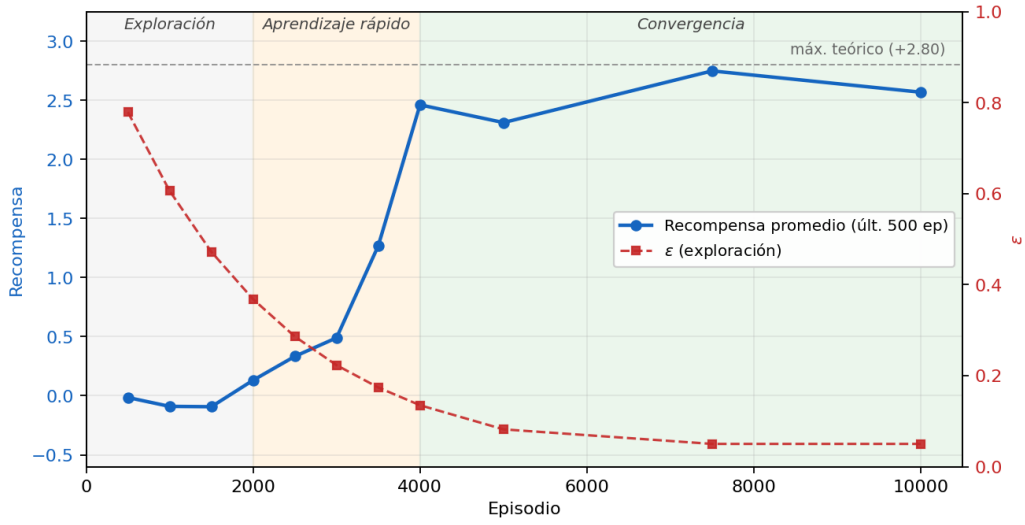


Figura 4: Recompensa promedio sobre los últimos 500 episodios (azul, eje izquierdo) y ϵ (rojo, eje derecho) en función del episodio. El sombreado de fondo separa las tres fases descritas en el texto. Datos del entrenamiento con $\alpha = 0.1$, $\gamma = 0.99$, $\epsilon : 1.0 \rightarrow 0.05$ y decaimiento 0.9995.

A. Trabajo adicional: laberinto rectangular 8×7

Este apéndice no forma parte de la respuesta a la rúbrica de la entrega parcial. La rúbrica se refiere exclusivamente al ambiente `MiniGrid-BlockedUnlockPickup-v0` descrito en el cuerpo principal. Lo incluimos como trabajo adicional desarrollado en paralelo, por trazabilidad con la discusión que se dio en el foro del curso.

A.1. Origen

En el foro del curso se planteó como ejemplo un laberinto rectangular 8×7 con muros internos en el que un agente debe ir de una celda inicial a una celda objetivo. Lo trabajamos en paralelo al ambiente principal, en parte por la discusión que se dio en el foro y en parte porque el laberinto es un caso limpio para validar la implementación: la solución óptima se obtiene de forma exacta con BFS, lo que da un punto de referencia objetivo para comparar la política aprendida por Q-learning.

A.2. Descripción

El laberinto es una grilla de 8 filas por 7 columnas (56 celdas en total) con 65 segmentos de muro entre celdas adyacentes (37 internos y 28 de borde). El agente parte en $(6, 0)$ y tiene que llegar a $(1, 6)$. La distancia Manhattan entre ambas casillas es 11, pero los muros internos fuerzan rodeos que llevan el camino mínimo a 25 pasos (verificado con BFS). El ambiente es determinista.

A.3. Estados y acciones

El estado es $s = (x, y)$ con $x \in \{0, \dots, 7\}$, $y \in \{0, \dots, 6\}$. Hay $|\mathcal{S}| = 56$. Las acciones son $\mathcal{A} = \{N, S, E, W\}$, todas siempre aplicables. Si la celda destino existe y no hay muro entre la celda actual y la destino, el agente se mueve. Si no, la acción es no-op: el agente se queda donde está y paga el costo del paso.

A.4. Recompensa

$$R(s, a, s') = \begin{cases} +100 & s' = (1, 6) \\ -1 & \text{en cualquier otro caso} \end{cases}$$

Es la formulación canónica de *gridworld* con costo de paso uniforme. Garantiza que la política óptima coincide exactamente con el camino más corto medido en pasos.

A.5. Resultados

Q-learning tabular con $\alpha = 0.1$, $\gamma = 0.99$, $\varepsilon : 1.0 \rightarrow 0.05$ y 2000 episodios produce una política greedy que resuelve los 100 episodios de evaluación en 25 pasos (igual al óptimo BFS) con retorno +76. La Figura 5 muestra el laberinto y el camino aprendido. La Figura 6 muestra la curva de aprendizaje.

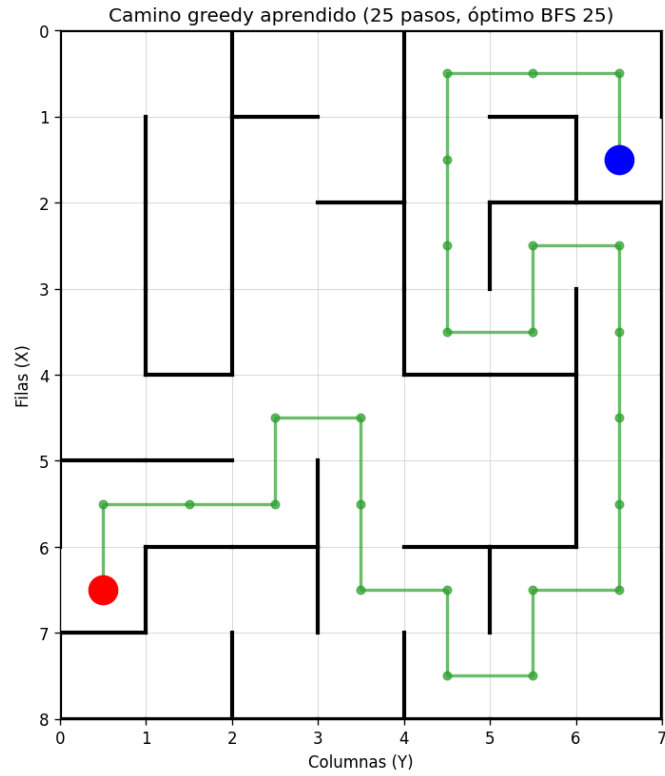


Figura 5: Laberinto y trayectoria greedy aprendida. Los 25 pasos coinciden con el óptimo de BFS.

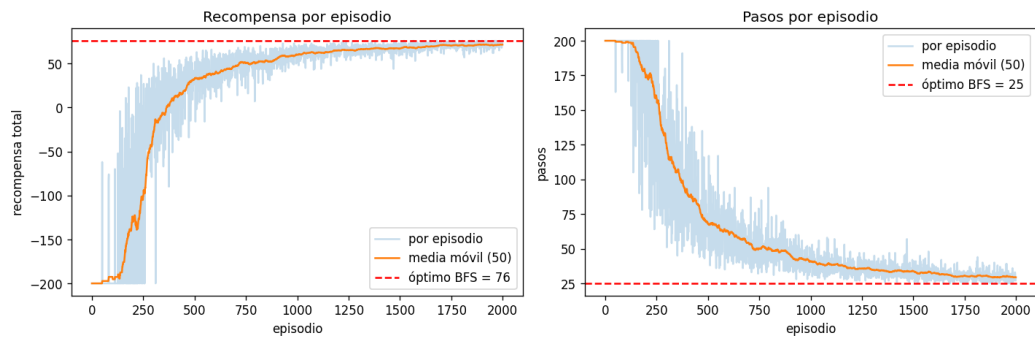


Figura 6: Curva de aprendizaje en el laberinto. El panel izquierdo muestra la recompensa por episodio; el derecho, el número de pasos por episodio.

Referencias

- [1] R. S. Sutton, A. G. Barto. *Reinforcement Learning: An Introduction*. 2.^a ed., MIT Press, 2018.
- [2] C. J. C. H. Watkins, P. Dayan. “Q-learning”. *Machine Learning*, vol. 8, n.º 3, 1992.
- [3] M. Chevalier-Boisvert, B. Dai, M. Towers, R. de Lazcano, L. Willems, S. Lahlou, S. Pal, P. S. Castro, J. Terry. “Minigrid & Miniworld: Modular & Customizable Reinforcement Learning Environments for Goal-Oriented Tasks”. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.